

ABRA — engineering review, 2026-07-31

Reviewed as **software a serious organisation would have to run**, not as research. The ideas are assumed sound. The question is whether the implementation is trustworthy.

Every claim here was produced by running something. Test quality was assessed by **mutation** — deliberately breaking the thing a suite claims to protect and confirming it fails. A suite that passes on broken code is worse than no suite, because it manufactures confidence.

Verdict: FIX THEN SHIP

This is not a rebuild candidate. The engineering is better than the project's own documentation claims — CI runs a *derived* test list, dependencies are locked, the simulator is pinned, and the two mutations I ran against the most load-bearing code were both caught.

It is also not shippable today, for one specific reason: `engine/validate_damage_sim.js` **is red, and has been red while nothing ran it**. That check exists to answer "are we driving the official simulator correctly", and it currently says no on 2 of 36 scenarios. Everything this project produces — every self-play game, every fitted weight, every published win rate — is downstream of that simulator being driven correctly.

If this ran in production and was wrong, the failure mode is not a crash. It is a **confidently reported win rate computed on a mis-driven engine**, which is the exact failure ADR-001 step 3 was written to prevent.

Findings, ranked by blast radius

1. The damage golden master was not in the test suite — and the meta-check missed it

`engine/validate_damage.js` validates this project's damage engine against `@smogon/calculator`, the independent ground truth. **It was not in** `GATES`.

`run-all.js` contains a coverage assertion written precisely to prevent this, whose own comment says "*a check that nothing runs is worse than no check — it reads as coverage in a review.*" It did not fire, because it detects a check **by its output format**:

```
const looksLikeACheck = src => /passed, .*failed/.test(src) || /console\.log\(' ok' /.test(src);
```

`validate_damage.js` reports an aggregate table and `process.exit(1)`. It never prints "N passed, N failed".

Matches: 0. The meta-check built to stop unrun checks was fooled by formatting, and the check it missed is the one guarding the number everything else depends on.

Proven by mutation. Neutering Sword of Ruin (a real 25% Defence cut → no-op):

	before	with mutant
within 5% of <code>@smogon/calculator</code>	100%	94% (needs ≥95)
worst error	0%	25% (needs ≤8)
full test suite	green	still green

The suite did not notice a real ability silently ceasing to function. `validate_damage.js` did.

FIXED — added to `GATES`; detector widened to recognise a gate by what it *does* (non-zero exit plus a regression announcement) rather than how it prints.

2. validate_damage_sim.js was also unrun, and it is RED

The widened detector immediately found a second unrun gate — the ADR-001 check that we are driving the **official Champions simulator** correctly. Its own header: *"A mis-wired simulator is indistinguishable from a working one unless something independent checks it."*

It fails:

```
Kingambit suckerpunch -> Gholdengo      calc 64-76      sim 64-152      worst 100%
Rillaboom closecombat -> Kingambit      calc 126-150    sim 126-300     worst 100%

within-5%: 97% (need >=95)   worst: 100% (need <=8)   -> exit 1
```

The **minimum agrees exactly** in both cases and the **maximum is exactly double**. That is systematic, not rounding.

NOT FIXED, deliberately. I do not understand the cause, and a damage discrepancy guessed at is worse than one left visibly red. It is now in the suite, failing loudly, which is the correct state for an unresolved correctness question. **This is the item blocking SHIP.**

3. The simulator pin was declared and never verified — and provenance lied by construction

champions_sim.js:49 pins Showdown to commit 20ad99ff... . **Nothing compared it to the checkout that actually loads.** Worse:

```
mew.js:716  rec.selfplay = { engine_commit: CS.PINNED_COMMIT, ... }
```

Every self-play record was stamped with **the constant**. A checkout at any other commit would still have produced games labelled 20ad99ff . The single field that records which engine generated a corpus was **unfalsifiable** — it could never disagree with the intent, so it could never detect a drift.

FIXED. actualCommit() reads HEAD from git; verify() now reports actual_commit and commit_matches (true / false / null **for unknown** — an unverifiable pin must never read as a verified one). mew.js stamps the real commit and keeps engine_pinned alongside so a mismatch is visible *in the data*. Verified: pinned 20ad99ffc9a5 , actual 20ad99ffc9a5 , matches true .

4. @smogon/calcul is a caret range

package.json declares "@smogon/calcul": "^0.11.0" — the **ground truth of the damage golden master** is free to move to 0.12 on a fresh install. package-lock.json exists and pins it for CI and for anyone running npm ci , which is the real mitigation. But a developer running npm install on a machine without the lock gets a different arbiter of truth, and the golden master would then be validating against a moved target while reporting the same numbers.

Not fixed — changing a dependency range is a decision with release implications, not a review's call. **Recommended:** pin exactly, and treat a @smogon/calcul bump as a deliberate re-baselining with the golden master re-run.

Test quality — measured by mutation, not by count

mutation	what it breaks	caught?	by what
behaviouralBots() → return new Set()	the bot filter — the project's single most load-bearing	YES	test-quality.js , 9 checks fail; clean count 5,129 → 6,736 (1,607 bot games leak in)

mutation	what it breaks	caught?	by what
	rule		
Sword of Ruin 0.75 → 1.00	a real ability silently stops working	YES	validate_damage.js — but only because I ran it manually; it was not in the suite

The two most important things in this codebase are genuinely protected. That is a real result and it is more than most projects of this size can show. The failure was not test *quality* — it was test *wiring*.

38 test files. After this review the suite discovers and runs the two gates it was missing.

Data integrity

The store-corruption history could **not** be verified from version control. `git log --grep` for duplication/corruption terms across all branches returns only ordinary feature commits. Either the incidents predate this history, were recorded only in prose, or the search terms are wrong.

I therefore cannot confirm the number of incidents or their causes, and I decline to restate the handoff note's figure as fact — the brief explicitly warns against that, and I have no independent evidence either way.

What I can verify, measured directly:

store	lines	unique ids	duplicates
games.ladder.jsonl	27,512	27,512	0
games.bo3.jsonl	5,175	5,175	0

quality.js opens a **named path**, not a glob, so experiment output cannot leak into the clean corpus. That is the structural property that matters and it is correct.

Operational maturity — better than the documentation suggests

	state
CI	3 workflows; tests.yml runs <code>node tests/run-all.js</code> — the list is <i>derived</i> , not typed. The 2026-07-27 hand-typed-list problem is genuinely fixed.
Lockfile	package-lock.json present
Simulator	pinned to an explicit commit, and now verified at runtime
Reproducible builds	md_to_pdf.js deliberately dependency-free (headless Chrome) because weasyprint/pandoc are absent on the author's machine
Rollback	git only. No tagged releases, no artifact versioning beyond <code>generated</code> fields.

Weakest link: rollback. There is no release to point at. "The current state of main" is the only deployable, and the site auto-updates from it.

Bus factor

Better than typical, with one structural risk.

The code is unusually well commented — not with *what* it does but with *why*, and specifically with **the failure that caused each decision**. That is the single most valuable property for a new engineer, and this codebase has it to an unusual degree.

The risk is that the documentation is load-bearing and was, until yesterday, four days stale. On 2026-07-30 that staleness caused a session to mischaracterise the whole model family — describing a built, measured model as unbuilt. A new engineer would have made the same error. It is now current, and `test-docs-current.js` gates it.

One-line summary a new engineer needs and can now get: `mew.js` → `magnemite.js` → `board.js` → `medicham2-browser.js` is the entire live path. Everything else is tooling.

What I would delete

target	why
23.7 GB of <code>games.h2h-*.jsonl</code> (30 files, up to 5.66 GB each)	dead experiment output; gitignored, so zero repo cost but real disk and it makes <code>provenance.js</code> glob 30 irrelevant files
<code>data/exploitability-machamp.json</code> , <code>exploitability-mag.json</code> , <code>exploitability.json</code>	three artifacts for one measurement, none declaring a source script; keep one, name it, generate it
<code>build/build_userscript.py</code> , <code>build/build_ui.py</code> (CHOMP)	superseded by the v2 builders; both still point at a sandbox path that has not existed for weeks

I would **not** delete SLOWKING, DODUO, or PORY despite none being in a live decision. Each is a fitted, measured artifact with a recorded negative or null result, and deleting them would destroy the evidence that the question was asked. That is the difference between sunk cost and a lab notebook.

Changes made in this review

file	change	verified
<code>tests/run-all.js</code>	<code>validate_damage.js</code> + <code>validate_damage_sim.js</code> added to <code>GATES</code>	suite now runs both
<code>tests/run-all.js</code>	coverage detector recognises exit-code gates	no false positives
<code>engine/champions_sim.js</code>	<code>actualCommit()</code> , <code>verify().commit_matches</code>	pinned/actual both <code>20ad99ffc9a5</code> , matches <code>true</code>
<code>engine/mew.js</code>	records stamp the real engine commit, plus <code>engine_pinned</code>	probe game carries both

Suite before: 38 passed, 1 failed. **After:** 39 passed, 2 failed — the new failure is `validate_damage_sim.js` , which was always failing and is now visible.

My own errors during this review

Recorded at the same severity as everything else.

- I widened the coverage detector to any `process.exit(1)` — which matched **36 engine files** doing ordinary error handling. That is crying wolf, the exact defect the assertion exists to prevent. Narrowed to require a regression announcement as well.
- I reported "NO LOCKFILE". `package-lock.json` exists; my `ls` exited non-zero only because `npm-shrinkwrap.json` does not, and the `||` fired. A shell artifact reported as a finding.

What I could not verify

- **The cause of the `validate_damage_sim` discrepancy.** Exactly-double maxima with matching minima on 2 of 36 scenarios. Left red rather than guessed at.
 - **The store-corruption incident count and cause.** Not recoverable from git history with the search terms available. Not restated from the handoff note.
 - **Whether `@smogon/calc` `0.12` would change the golden master's verdict.** Would require installing it, which changes the ground truth mid-review.
 - **The 31 artifacts `provenance.js --strict` calls UNSAFE.** Confirmed by the systems audit as "unprovable from disk"; settling it requires re-running 31 generators.
-

To keep my confidence, the team must

1. **Resolve `validate_damage_sim`.** Not silence it. It is 2 scenarios out of 36 and the pattern is specific enough to be findable.
 2. **Pin `@smogon/calc` exactly,** and treat a bump as a deliberate re-baselining.
 3. **Wire `provenance.js --strict` into the suite,** or explicitly waive the 31 artifacts by name. The systems audit found the same shape; two independent reviews finding the same unwired gate is the signal.
 4. **Produce one tagged release.** There is currently nothing to roll back to.
-

Reviewed 2026-07-31 against commit `4134a83`. The self-play training run completed at 04:28 before mutation testing began, so the whole repository was safe to mutate; all mutants were restored and verified. PDF built with `build/md_to_pdf.js` rather than `build/omnibus.py`, because omnibus renders via weasyprint and this machine has neither weasyprint nor pandoc on the path — the reason `md_to_pdf.js` exists.